

PRD — AI Knowledge Assistant

Project Overview

AI Knowledge Assistant ek RAG (Retrieval-Augmented Generation) based intelligent document assistant hoga jo PDFs aur multiple documents ko read karke user ke natural language questions ka accurate answer dega with source citations.

Yeh project Intimetec Hackathon assignment ke according design kiya gaya hai jahan focus hai:

- LLM integration
 - RAG architecture
 - Semantic search
 - Scalable AI system design
 - Production-level thinking
-

Problem Statement

Traditional keyword search systems semantic meaning understand nahi karte.

Example:

Document: "Employees can work remotely twice a week"

User Query: "What is company work from home policy?"

Traditional search fail ho sakta hai because exact keywords match nahi hue.

AI Knowledge Assistant:

- query ka meaning samjhega
 - relevant context retrieve karega
 - accurate AI-generated answer dega
 - source citation show karega
-

Project Goal

Ek intelligent enterprise-grade document assistant build karna jo:

- multiple documents handle kare

- semantic understanding use kare
 - conversational question answering support kare
 - source citations provide kare
 - scalable architecture follow kare
-

Core Features

1. PDF & Document Upload

Users multiple documents upload kar sakte hain.

Supported formats:

- PDF
 - TXT
 - DOCX (optional)
-

2. AI Question Answering

User natural language mein question poochega.

Example:

- What is refund policy?
- What are employee leave rules?
- What is deployment process?

AI relevant information retrieve karke answer generate karega.

3. Semantic Search

Keyword-based nahi.

Meaning-based search use hoga.

Example: "refund" aur "money back" related understand honge.

4. Source Citations

Answer ke niche source document aur chunk reference show hoga.

Example:

- HRPolicy.pdf
 - Page 4
-

5. Chat Interface

Simple conversational UI.

Features:

- Question input
 - AI response area
 - Source display
 - Upload section
-

Recommended Tech Stack

Frontend

Next.js

Reason:

- Fast development
- Modern UI ecosystem
- Easy deployment

Tailwind CSS

Reason:

- Rapid UI building
- Clean modern design

Shadcn UI

Reason:

- Professional UI components
 - Minimal setup
-

Backend

FastAPI (Python)

Reason:

- Best ecosystem for AI/ML
 - Fast APIs
 - LangChain integration easy
-

AI Stack

LLM

Gemini API

Reason:

- Cheap
- Fast
- Strong context handling

Alternative:

- OpenAI GPT-4
-

Embedding Model

all-MiniLM-L6-v2

Reason:

- Lightweight
 - Fast
 - Free
 - Strong semantic embeddings
-

Vector Database

ChromaDB

Reason:

- Easy setup
- Free
- Good for hackathon MVP

Alternative:

- Pinecone
-

Framework

LangChain

Reason:

- RAG pipeline easy
 - Document loaders
 - Retrieval system
 - LLM orchestration
-

System Architecture

Complete Flow

1. User uploads documents
 2. Backend extracts text
 3. Documents split into chunks
 4. Embeddings generated
 5. Stored inside vector database
 6. User asks question
 7. Semantic retrieval runs
 8. Relevant chunks fetched
 9. LLM receives context
 10. AI generates answer
 11. Sources displayed
-

Architecture Components

Document Loader

Purpose: Extract text from PDFs and documents.

Libraries:

- PyPDFLoader
 - Unstructured
-

Chunking System

RecursiveCharacterTextSplitter

Configuration:

- Chunk Size: 500–1000
- Chunk Overlap: 100–200

Reason:

- Better context retention
 - Improved retrieval accuracy
-

Embedding Pipeline

Text chunks converted into vector embeddings.

Purpose: Semantic similarity search.

Vector Store

Stores embeddings for fast retrieval.

Database:

- ChromaDB
-

Retrieval Layer

Top-k relevant chunks retrieve karega.

Recommended:

- Similarity Search
 - Hybrid Search (optional)
 - Reranking (future improvement)
-

LLM Response Layer

Retrieved context LLM ko diya jayega.

LLM:

- Gemini

Output:

- Context-aware answer
 - Source citations
-

Folder Structure

```
project/
|
├── frontend/
|   ├── components/
|   ├── pages/
|   ├── app/
|   └── styles/
|
├── backend/
|   ├── routes/
|   ├── services/
|   ├── rag/
|   ├── embeddings/
|   ├── vectorstore/
|   └── main.py
|
└── uploads/
```

└─ chroma_db/
└─ README.md

API Endpoints

Upload Documents

POST /upload

Purpose: Upload PDFs and process embeddings.

Ask Questions

POST /ask

Purpose: Receive user query and generate AI response.

UI Screens

1. Home Screen

- Hero section
 - Upload button
 - Chat panel
-

2. Chat Interface

- Question input
 - AI answers
 - Sources
 - Loading animation
-

3. Document Panel

- Uploaded file list
 - File status
-

Biggest Failure Mode

Hallucination

Problem: LLM confident but incorrect answer de sakta hai.

Mitigation Strategy

- Source citations mandatory
 - Retrieval filtering
 - Top-k optimization
 - Low temperature responses
 - Better chunking
 - Context grounding
-

MVP Scope (2-Day Build)

Must Build

- PDF upload
 - Text extraction
 - Chunking
 - Embeddings
 - ChromaDB integration
 - Question answering
 - Source citations
 - Simple UI
-

Optional Features

Build only if time remains:

- Authentication
- Streaming responses
- Multi-user support
- Dark mode
- Chat history
- DOCX support
- Hybrid retrieval

Day-wise Execution Plan

Day 1

Backend

- FastAPI setup
- PDF upload
- Text extraction
- Chunking pipeline
- Embeddings generation
- ChromaDB setup
- Retrieval testing

Frontend

- Next.js setup
- Basic UI
- Upload component
- Chat component

Day 2

AI Integration

- Gemini integration
- RAG pipeline connection
- Source citations

Finalization

- UI polish
 - README
 - Deployment
 - Demo video
 - GitHub cleanup
-

Deployment Strategy

Frontend

Deploy on:

- Vercel

Backend

Deploy on:

- Render
 - Railway
-

Success Criteria

Project successful tab mana jayega agar:

- User documents upload kar sake
 - AI accurate answers de
 - Semantic search work kare
 - Sources visible ho
 - UI functional ho
 - GitHub clean ho
 - Demo properly work kare
-

Final Deliverables

Required:

- GitHub repository
 - README setup guide
 - Live demo (optional but strong advantage)
 - Short demo video
 - Architecture explanation
-

Final Project Summary

AI Knowledge Assistant ek RAG-powered enterprise document intelligence system hai jo company documents ko understand karke users ke questions ka accurate, context-aware aur citation-supported answer provide karta hai using semantic search and LLMs.